



Frontend Intern Challenge: Next-Gen Learning Dashboard

Context

We are building a futuristic, highly animated education platform. We need engineers who understand that "new edge" design isn't just about flashy visuals — it's about hardware-accelerated animations, zero layout shifts, and a buttery-smooth user experience powered by efficient, server-rendered data.

Your task is to build a high-fidelity "Student Dashboard" prototype that fetches live data from Supabase.

1. The Layout & Architecture

Theme: Dark mode only. The UI should rely on deep background tones (e.g., near-blacks and dark grays) accented by subtle, glowing gradients.

Structure: Implement a **Bento Grid** layout.

The dashboard must be split into two main sections:

- **Sidebar (Left):** A slim, collapsible navigation menu.
- **Main Content (Right):** A dynamic grid containing the following "Bento" tiles:
 - **Hero Tile:** A large greeting tile ("Welcome back, [Name]") with a daily learning streak indicator.
 - **Course Tiles (Dynamic):** Smaller tiles displaying active courses fetched from your database.
 - **Activity Tile:** A medium tile showing a mock contribution graph or activity chart.

2. The Tech Stack & Constraints

You must use the following tools. Substitutions are not permitted for this assignment.

- **Framework:** Next.js (App Router required).
- **Database/BaaS:** Supabase.
- **Styling:** Tailwind CSS.
- **Animations:** Framer Motion (Strict requirement).
- **Icons:** Lucide React or Radix Icons.

Strict Constraints:

- **No "div soup":** Use semantic HTML elements (e.g., <nav>, <main>, <article>, <section>).
- **Zero Layout Shifts:** Hover states and entrance animations must not trigger browser repaints or layout shifts. Use transform and opacity exclusively for animations.
- **Component Modularity:** Break the UI down into sensible, reusable components.

3. Data Integration (Supabase + Server Components)

Instead of hardcoding the course data, you must fetch it from a Supabase PostgreSQL database.

1. **Database Setup:** Create a free Supabase project and set up a courses table with the

following schema:

- id (uuid, primary key)
- title (text) - e.g., "Advanced React Patterns"
- progress (integer) - e.g., 75
- icon_name (text) - A string representing the Lucide icon to render.
- created_at (timestamp)

2. **Seed Data:** Insert 3-4 mock rows into your Supabase table.

3. **Data Fetching:**

- You MUST fetch this data using **Next.js Server Components** (RSC).
- Use the @supabase/ssr package (or @supabase/supabase-js) to connect securely from the server.
- **Loading States:** Implement a loading.tsx or use React <Suspense> boundaries to show skeleton loaders (which should also have a subtle pulsing animation) while the Supabase data is being fetched.
- **Error Handling:** Implement graceful error handling if the database connection fails.

4. Animation & Interaction Requirements

The "feel" of this app is just as important as the look. We expect the following interactions handled via Framer Motion:

- **Staggered Page Load:** Once the data is fetched and rendered, the Bento tiles should not appear all at once. They must stagger in sequentially (e.g., fading in while translating slightly upward on the Y-axis).
- **Card Hover States:**
 - When hovering over a Bento tile, it should elevate slightly (scale up by ~1-2%).
 - Reveal a subtle border glow or gradient shift on hover.
 - *Requirement:* Use Framer Motion's spring physics (e.g., type: "spring", stiffness: 300, damping: 20) for a natural, non-linear feel.
- **Micro-interactions:** Sidebar navigation items should feature a background highlight that snaps into place using layout animations (layoutId in Framer Motion) when clicked.

5. Course Card Specifications

Each of the dynamic course tiles fetched from Supabase must contain:

1. **Icon:** Dynamically rendered based on the icon_name field from the database.
2. **Title:** The course name.
3. **Progress Indicator:** A custom animated progress bar. On initial load, the bar should animate from 0% to the progress value fetched from the database.
4. **Background:** A very subtle, abstract gradient mesh or grain texture that sits behind the card content.

6. Responsive Design

- **Desktop (> 1024px):** Full Bento grid with the sidebar visible.
- **Tablet (768px - 1024px):** Sidebar collapses to icons only. Bento grid adjusts to a

2-column layout.

- **Mobile (< 768px):** Sidebar moves to a bottom navigation bar or hamburger menu. The Bento grid stacks into a single, vertical scrolling column.



Evaluation Rubric

We will review your code based on the following criteria:

1. **Data Architecture & Next.js (30%):** Did you successfully implement Server Components for data fetching? Are your Supabase environment variables handled securely? Did you implement effective Suspense/loading skeletons?
2. **Framer Motion Proficiency (30%):** Are the animations performant? Did you use spring physics correctly? Are you avoiding layout shifts?
3. **Code Quality & Types (20%):** Is your component tree logical? Are you using TypeScript interfaces for your Supabase data payloads?
4. **Visual Fidelity & Responsiveness (20%):** Does the final product look like a premium, modern application? Does the layout degrade gracefully on mobile?

How to Submit

1. Host your code on a public GitHub repository.
2. Deploy the application using Vercel.
3. Include a .env.example file so we know which Supabase environment variables are required. Do NOT commit your actual .env file.
4. Include a brief README.md explaining your architectural choices, how you handled the server/client component split, and any challenges you faced.